CS 4361
Assignment 4: Shaders

Due 11:59PM, November 10, 2026

1 Introduction

In this assignment, you will utilize your knowledge of lighting and shading on 3D models. Here we will study how to implement various shading algorithms: Blinn-Phong, Toon, PBR (Physically Based Rendering) as well as some other fun shaders. This is a rather interesting assignment, so we hope you will have fun with it!

1.1 Getting the Code

The assignment code is released on the e-learning page. Please download it to your local machine, navigate to the folder where you intend to keep your assignment code, and run the following command from the terminal or command line:

```
unzip a4-release.zip
```

1.2 Template

- `A4.html` is the launcher of the assignment. Open it in your preferred browser to run the assignment, to get started. You do not need to modify this file.
- `A4.js` contains the JavaScript code used to set up the scene and the rendering environment. You need to modify it for question d.
- `glsl/` contains the vertex and fragment shaders for the dragon and the light sphere. You need to modify the Blinn-Phong, toon and dots shaders.
- `js/` contains the required JavaScript libraries. Generally, you do not need to change anything here unless explicitly asked.
- `obj/` contains the dragon model used in scenes 1–3.
- `gltf/` contains the helmet model used in scene 4, as well as the texture images to be applied to it.

2 Work to be done (100 points)

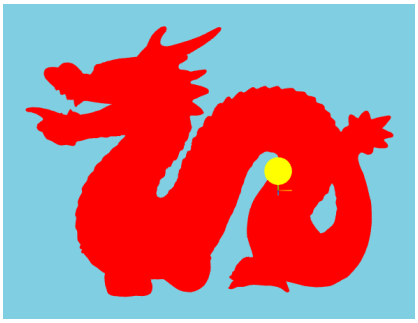


Figure 1: Initial configuration

Here we assume you already have a working development environment which allows you to run your code from a local server (if you do not, check out the instructions from Assignment 1 for details). The initial scene should look as in Figure 1. Once you have set up the environment, study the template to get a sense of what values are passed to each shader file and how. There are four scenes, and you may toggle between them using the number keys 1, 2, 3, and 4: 1 - Blinn-Phong, 2 - Toon, 3 - Polka dots, 4 - Three.js PBR. The light sphere can be moved with the W/A/S/D keys (on the x-z plane) and the Q/E keys (up and down). The default scene is set to 1. See `let mode = shaders.BLINNPHONG.key;` in `A4.js`. You may find it convenient during your development to change this default value to the scene containing the shader that you are currently working on (e.g. `let mode = shaders.TOON.key;` for question b).

2.1 Part 1: Required Features

a. **(25 points)** Scene 1: Phong Reflection

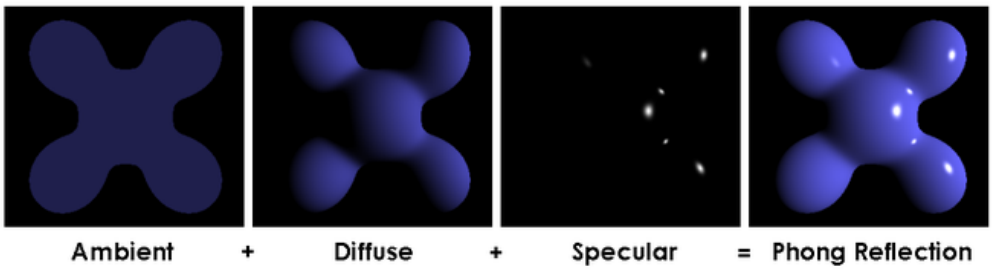


Figure 2: Phong reflection model.

First of all, note that the Phong reflection model is a different type of thing than the Phong shading model; they just happen to be named after the same person. The latter improves on the Gouraud shading by computing the lighting per fragment, rather than per vertex. This is done by using the interpolated values of the fragment's position and

normal. In this scene, you will implement the Blinn-Phong version of the Phong reflection model. Figure 2, taken from the Wikipedia article on this model, shows how different components look either individually or when summed together.

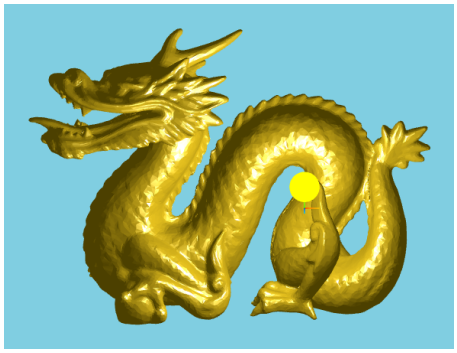


Figure 3: Question a: Blinn-Phong Dragon.

Complete the code in `blinn_phong.vs.glsl` and `blinn_phong.fs.glsl` to shade the Dragon in Scene 1 using the Blinn-Phong reflection algorithm. While the majority of computation should happen in the fragment shader, you still need to add some code to the vertex shader in order to pass appropriate information to your fragment shader. Your resulting Dragon should look like the one shown in Figure 3.

b. **(25 points)** Scene 2: Toon

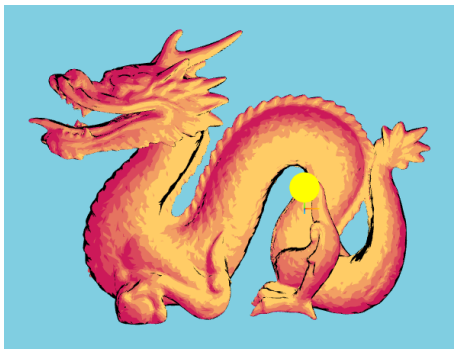


Figure 4: Question b: Toon Dragon.

Send the Dragon to the realm of action and superheroes! Unlike the smooth, realistic shading in the previous question, Toon shading gives a non-photorealistic result. It emulates the way cartoons use very few colors for shading, and the color changes abruptly, while still providing a sense of 3D for the model. This can be implemented by quantizing light intensity across the surface of the object. Instead of making the intensity vary smoothly, you should quantize this variation into a number of steps for each "layer" of toon shading. Use two tones to colour the Dragon; red for the darker areas (lower light intensity) and yellow for the lighter areas (higher light intensity), as shown in Figure 4.

Lastly, draw dark silhouette outlines on the Dragon. Complete the code in `toon.vs.glsl` and `toon.fs.glsl`.

Hint 1: The two tones are most easily done by interpolating between two predefined colours. For the outlines, you can determine for each fragment whether it is a part of the object's outline by computing the cosine of the angle between the fragment's normal and the viewing direction: fragments that are "edgy" enough should be outlines. If you need some inspiration, the following video games were rendered with toon shading (also called cel shading) techniques:

https://en.wikipedia.org/wiki/List_of_cel-shaded_video_games

Hint 2: Since the silhouette is determined using the surface normal and the viewing direction, and does not depend on the light direction, moving the light source should not affect its location.

c. **(30 points)** Scene 3: Polka Dots

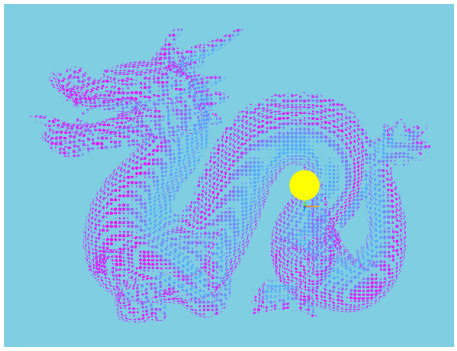


Figure 5: Question c: Polka-dotted Dragon.

The possibilities are endless! Here, we ask you to (i) give the Dragon polka dots by checking whether a fragment is close enough to points on a regular grid in the 3D space around the Dragon; (ii) make the polka dots smoothly "roll" down the body over time; and (iii) make the dots smoothly change colour. To implement these effects, you will need to modify `dots.vs.glsl` and `dots.fs.glsl`, so the fragment shader shades the fragment depending on the time `ticks` and the local vertex position, and discards fragments that you don't want to render: see the empty space between the dots. The final result should look like what is shown in Figure 5. If you are unable to implement rolling dots, you can still get partial marks for completing (iii).

Hint 1: Make use of some oscillatory mathematical function(s) to implement the color change.

Hint 2: The `discard` statement in GLSL throws away the current fragment, so it is not rendered.

Hint 3: You may find the GLSL `mod()` function to be useful. It computes the modulus of two floats. Example: `mod(0.3, 0.2) = 0.1`.

Hint 4: The Dragon's model coordinates may be smaller than you think. If your Dragon disappears completely (not because of some syntax error) try using smaller numbers when computing regions to discard.

d. **(20 points)** Scene 4: Three.js PBR and texturing

In this part, we'll get our first look at how to texture an object using Three.js, and to create a physically based rendering of a damaged, sci-fi themed helmet.

Many objects have many small details and facets of the surface (e.g., wood grain, scratches on metal, freckles on skin). These are very difficult, if not impossible, to model as a single material lit by a Phong-like model. In order to efficiently simulate these materials we usually use texture mapping. In basic texture mapping, UV coordinates are stored as vertex attributes in the vertex buffer (Three.js provides them in the *`vec2`* `uv` attribute for default geometries such as planes and spheres). UV coordinates allow you to look up sampled data, such as colors or normals, stored in a texture image, as discussed in class. Figure 6 shows the textures we will use for rendering the damaged helmet.

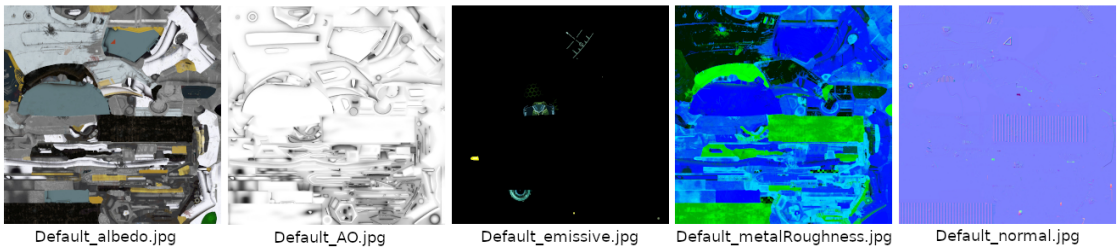


Figure 6: Textures used for rendering the damaged helmet.

PBR or Physically Based Rendering is a technique that simulates the interaction of light with materials in a realistic manner, considering physical properties such as the reflectance (or albedo), roughness, and metalness to achieve lifelike visual representations. This information is stored in a set of texture images. For more information, you can take a look at this great writeup: `https://learnopengl.com/PBR/Theory`.



Figure 7: Question d: PBR with `MeshStandardMaterial`.

Your task is to apply the textures (`.jpg` files) in `gltf/` to `helmetPBRMaterial`, which is already defined as Three.js's built-in type `MeshStandardMaterial`, and to set its metalness. For this, modify `A4.js`. The result should look like Figure 7.

Textures in GLSL are specified as `sampler2D` uniforms, and the values can be looked up using the `texture()` function; Three.js' built-in materials can do this for you. You need to load the relevant textures using `THREE.TextureLoader`. You can consult the Three.js documentation for more information.

Hint 1: `Default_metalRoughness.jpg` stores roughness in its green channel and metalness in its blue channel, which is what `roughnessMap` and `metalnessMap` read, so the same texture can be passed to both. An `emissiveMap` is multiplied by the material's `emissive` color, which is black by default.

Hint 2: These textures follow the glTF conventions rather than Three.js's defaults: do not flip them when loading (`flipY = false`), and let them repeat (`THREE.RepeatWrapping`), since the helmet's UV coordinates lie outside $[0, 1]$. The color textures (albedo and emissive) are in sRGB (`THREE.SRGBColorSpace`).

2.2 Part 2: Creative License (Optional)

You have many opportunities to unleash your creativity in computer graphics! In this **optional** section, you are invited to extend the assignment in fun and creative ways. We'll highlight some of the best work in class. A small number of exceptional contributions may be awarded bonus points. Here are a couple of ideas we can think of:

- Write your own PBR shader (e.g., Cook-Torrance) in a `ShaderMaterial` for the helmet and compare it with `MeshStandardMaterial`.
- Add another non-photorealistic style, such as hatching or more toon bands.
- Combine shaders in fun ways, e.g. polka dots that are lit with Blinn-Phong.

3 Submission Instructions

3.1 Directory Structure

You must submit your final code and write a clear `README` file which includes your name, student number, and the core idea/changed code/explanation/screenshot of each question. The TAs strongly encourage you to read the template of answers in `template.md` and use a similar template to submit your answers.
Your README file can be in PDF, Word, or Markdown format. It must be placed in the root directory and will serve as the basis for grading your work. **Missing a README file will result in a 30-point deduction.**

3.2 Submission Methods

Please compress everything under the root directory of your assignment into `a4.zip` and submit it on E-learning. You can make multiple submissions, but we will grade only the last one.

4 Grading

4.1 Point Allocation

Each assignment has 100 points for Part 1. Part 2 is optional, and you can get bonus points (0-10 points) at the instruction team's discretion. The max score for each assignment is 110 points. **Missing a README file will result in a 30-point deduction.**

4.2 Grading

The TA will carefully review your README file and evaluate based on the idea/code/ explanation/screenshot. If the program shows correct rendering effects, full points will be graded.

4.3 Anti-AI/Plagiarism Measures

To prevent plagiarism, 15 students will be randomly selected for face-to-face grading. These students are required to present their final submitted code to the instructor during office hours and point out the locations of core code for the corresponding problems.

4.4 Penalties

Aside from penalties for incorrect solution or plagiarism, we may apply the following penalties to each assignment:
Late penalty. A deduction of 10 points would be applied to each extra late day. Note that

1. We check the time of your last submission to determine if you are late or not;
2. We do not consider Part 1 and Part 2 submissions separately. Say if you submitted Part 1 on time but updated your submission for Part 2 one day after the deadline, that counts as one late day.

AI/Plagiarism penalty. If a student cannot point out where their code is located for solving specific problems (e.g., which lines of GLSL code correspond to the answer for a particular problem) and cannot clearly articulate their approach to solving the problem, this will result in a complete deduction of points for the corresponding assignment.